DevOps Project Report:
Scientific Calculator

Name: Chahak Agarwal
RollNo: MT2025031

---

1. Introduction to DevOps

DevOps is a collection of flexible practices and processes organizations use to create and deliver applications and services by aligning and coordinating software development with software processes. It uses practices like Continuous Integration(CI) , Continuous Deployment (CD), and automation to reduce errors and improve efficiency in the software development lifecycle.

DevOps help organizations deliver applications quickly ,improve software quality, and respond faster to user needs by enabling better collaboration between development and operations teams.

This project focuses on designing and deploying a Scientific Calculator application using modern DevOps practices.The application is built using Java and supports the following operations:

- **Square root function (√x)**
- **Factorial Function (!x)**
- **Natural Logarithm (ln(x))**
- **Power function (x^b)**

2. Tools Used

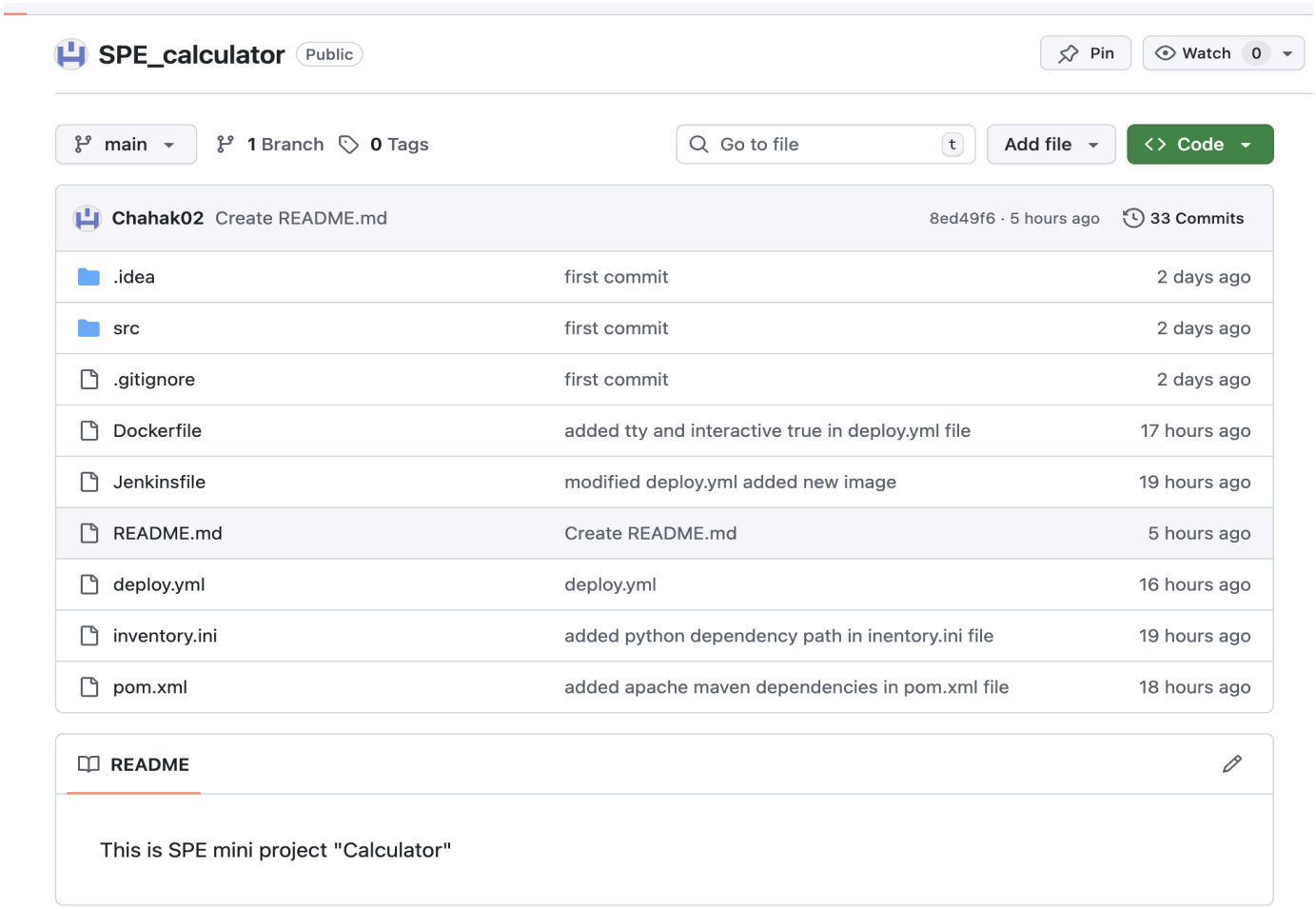
Tool	Purpose
GitHub	Source Code Management
JUnit	UnitTesting
Maven	Build Automation
Jenkins	Continuous Integration and Delivery (CI/CD)
Docker	Containerization
DockerHub	Image Registry
Ansible	Automated Deployment

3. Project Setup

Git Commands

- git clone https://github.com/Chahak02/SPE_calculator.git
- git add .
- git commit -m “changes or files added”
- git push origin main

Screenshot 1: (GitHub Repository)



4.Testing With JUnit

- Junit is used to perform unit testing for the scientific calculator functions to ensure that each operations work correctly
- Test cases were written for functions such as power, factorial ,natural log and square root to verify both valid and invalid inputs.
- JUnit assertions like assertEquals and assertThrows were used to check the correctness of result and proper handling of exceptions.

Code Snippet : CalciTest.java

```java
public class CalciTest {  Chahak02
    //  squareRoot Tests
    @Test  Chahak02
    void testSquareRootValid() { assertEquals( expected: 5.0, Calci.squareRoot( n: 25),  delta: 0.0001); }

    @Test  Chahak02
    void testSquareRootNegativeInput() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calci.squareRoot( n: -10);
        });
    }

    //  factorial Tests
    @Test  Chahak02
    void testFactorialValid() {
        // Expected factorial: 5! = 120
        assertEquals( expected: 120, Calci.factorial( n: 5));
        assertEquals( expected: 1.0,Calci.factorial( n: 0));
    }

    @Test  Chahak02
    void testFactorialNegative() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calci.factorial( n: -1);
        });
    }

    @Test  Chahak02
    void testFactorialTooLarge() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calci.factorial( n: 25);
        });
    }
```

```java
    //  naturalLog Tests
    @Test  Chahak02
    void testNaturalLogValid() { assertEquals(Math.log(10), Calci.naturalLog( n: 10),  delta: 0.0001); }

    @Test  Chahak02
    void testNaturalLogZero() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calci.naturalLog( n: 0);
        });
    }

    @Test  Chahak02
    void testNaturalLogNegative() {
        assertThrows(IllegalArgumentException.class, () -> {
            Calci.naturalLog( n: -5);
        });
    }

    // Power Tests
    @Test  Chahak02
    void testPowerValid() { assertEquals( expected: 32.0, Calci.power( base: 2,  exp: 5),  delta: 0.0001); }

    @Test  Chahak02
    void testPowerZeroExponent() { assertEquals( expected: 1.0, Calci.power( base: 10,  exp: 0),  delta: 0.0001); }
}
```

Screenshot: JUnit Test Results

```
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ Calculator ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running CalciTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in CalciTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ Calculator ---
[INFO] Building jar: /Users/veeralagarwal/Documents/MTECH_2/SPE/Calci/Calculator/target/Calculator-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.442 s
[INFO] Finished at: 2026-03-06T14:57:28+05:30
[INFO] ------------------------------------------------------------------------
```

5. Build With Maven

pom.xml configuration

- Added dependencies for JUnit and Maven Plugin
- Configured <mainClass> for executable JAR

Build Command :

mvn clean package

Screenshot 3: Maven Build Output

```
[INFO] Scanning for projects...
[INFO]
[INFO] --------------------< spe.calculator:Calculator >---------------------
[INFO] Building Calculator 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ Calculator ---
[INFO] Deleting /Users/veeralagarwal/Documents/MTECH_2/SPE/Calci/Calculator/target
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ Calculator ---
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
```

```
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ Calculator ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 2 source files with javac [debug target 17] to target/classes
[WARNING] location of system modules is not set in conjunction with -source 17
  not setting the location of system modules may lead to class files that cannot run on JDK 17
    --release 17 is recommended instead of -source 17 -target 17 because it sets the location of system modules automatically
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ Calculator ---
[INFO] skip non existing resourceDirectory /Users/veeralagarwal/Documents/MTECH_2/SPE/Calci/Calculator/src/test/resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ Calculator ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug target 17] to target/test-classes
[WARNING] location of system modules is not set in conjunction with -source 17
  not setting the location of system modules may lead to class files that cannot run on JDK 17
    --release 17 is recommended instead of -source 17 -target 17 because it sets the location of system modules automatically
[INFO]
```

```
[INFO]
[INFO] --- surefire:3.2.5:test (default-test) @ Calculator ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running CalciTest
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in CalciTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ Calculator ---
[INFO] Building jar: /Users/veeralagarwal/Documents/MTECH_2/SPE/Calci/Calculator/target/Calculator-1.0-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.442 s
[INFO] Finished at: 2026-03-06T14:57:28+05:30
[INFO] ------------------------------------------------------------------------
```

6. Continuous Integration With Jenkins

Jenkins Pipeline

Stages: Checkout →Build→Test→Docker build/push →Ansible Deployment

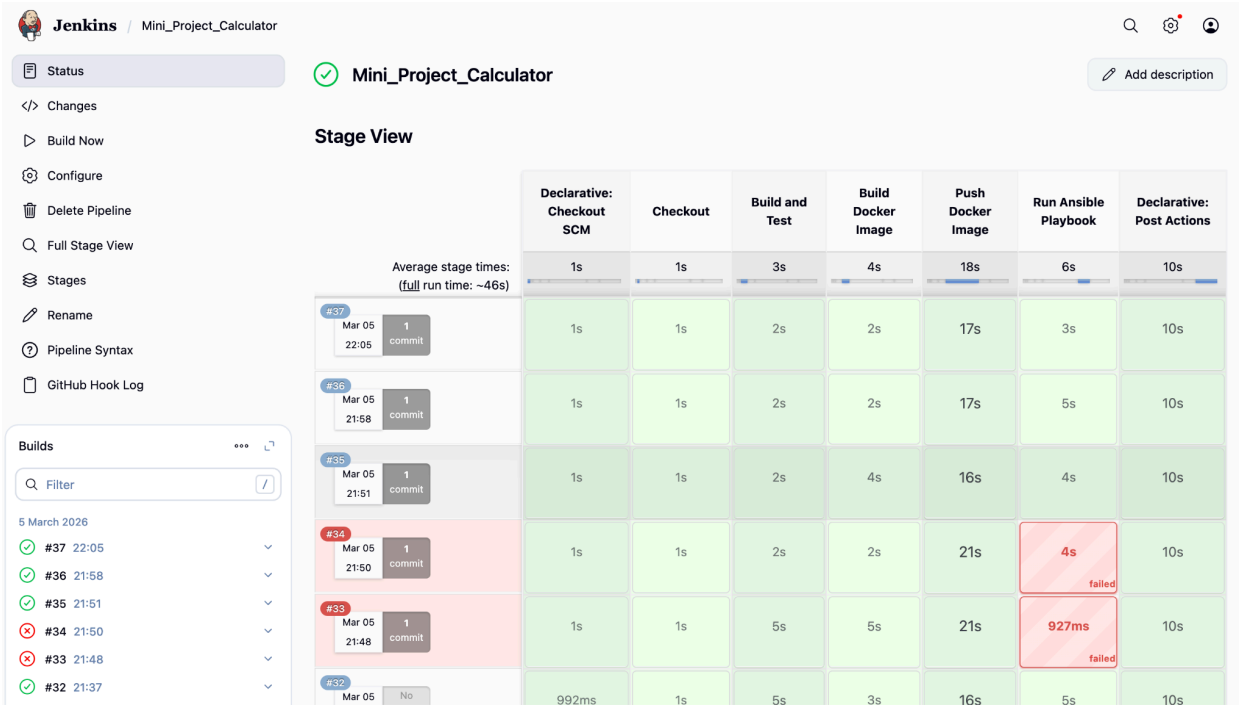
Screenshots:

```
pipeline {
    agent any
    environment {
        DOCKER_IMAGE_NAME = 'calculator'
        GITHUB_REPO_URL = 'https://github.com/Chahak02/SPE_calculator.git'
        DOCKER_REGISTRY = 'docker.io'
        DOCKER_USERNAME = 'chahak02'
        IMAGE_TAG = '0.0.1'
    }
    stages {
        stage('Checkout') {
            steps {
                script {
                    git branch: 'main', url: "${GITHUB_REPO_URL}"
                }
            }
        }
        stage('Build and Test') {
            steps {
                script {
                    sh 'mvn clean install'
                }
            }
        }
        stage('Build Docker Image') {
            steps {
                script {
                    // docker.build("${DOCKER_IMAGE_NAME}:${IMAGE_TAG}", '.')
                    // // Optionally, also tag the image as "latest" for consistency
                    // docker.build("${DOCKER_IMAGE_NAME}:latest", '.')
                    def app = docker.build("${DOCKER_IMAGE_NAME}:${IMAGE_TAG}")
sh "docker tag ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} ${DOCKER_IMAGE_NAME}:latest"
                }
            }
        }
```

```
        stage('Push Docker Image') {
    steps {
        withCredentials([usernamePassword(
            credentialsId: 'DockerHubCred',
            usernameVariable: 'USER',
            passwordVariable: 'PASS'
        )]) {

            sh 'echo $PASS | docker login -u $USER --password-stdin'

            sh "docker tag ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} ${DOCKER_USERNAME}/${DOCKER_IMAGE_NAME}:${IMAGE_TAG}"
            sh "docker tag ${DOCKER_IMAGE_NAME}:${IMAGE_TAG} ${DOCKER_USERNAME}/${DOCKER_IMAGE_NAME}:latest"

            sh "docker push ${DOCKER_USERNAME}/${DOCKER_IMAGE_NAME}:${IMAGE_TAG}"
            sh "docker push ${DOCKER_USERNAME}/${DOCKER_IMAGE_NAME}:latest"
        }
    }
}

        stage('Run Ansible Playbook') {
            steps {
                script {
                    ansiblePlaybook(
                        playbook: 'deploy.yml',
                        inventory: 'inventory.ini',
                         extras: "-e tag=${IMAGE_TAG}"
                        // extras: '-K'
                    )
                }
            }
        }
    }
```

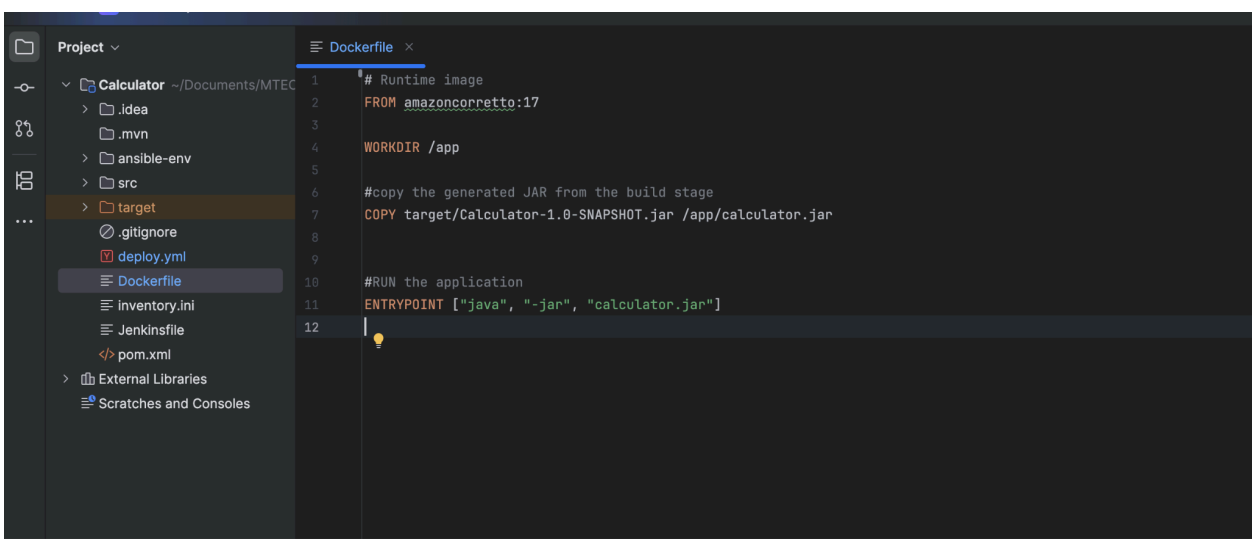
```
    post {
        success {
            echo 'Pipeline successfully completed!'
            emailext(
                to:'chahakagarwal02@gmail.com',
                subject: 'Build success: Scientific Calculator',
                body: 'The Jenkins pipeline for the Scientific calculator project has completed successfully.'
            )
        }
        failure {
            echo 'Pipeline failed!'
            emailext(
                to:'chahakagarwal02@gmail.com',
                subject: 'Build failure: Scientific Calculator',
                body: 'The Jenkins pipeline for the Scientific calculator project has failed.'
            )
        }
    }
}
```

Screenshots 4: Jenkins Pipeline



7. Containerization With Docker

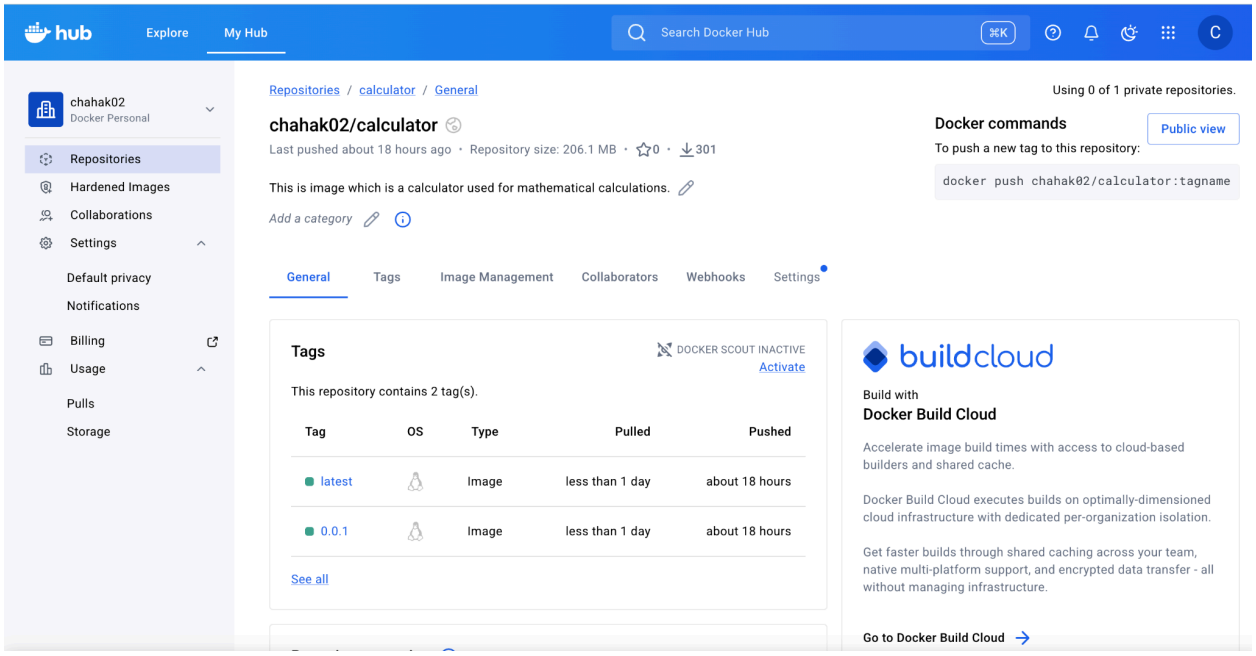
Dockerfile



Docker Commands:

- Docker build -t chahak02/calculator:0.0.1 .
- Docker push chahak02/calculator:0.0.1

Screenshot: Docker Image On Docker Hub



8. Deployment with Ansible

Ansible playbook(deploy.yml)

```yaml
---
- name: Deploy Calculator
  hosts: localhost
  connection: local

  vars:
    image_name: chahak02/calculator
    image_tag: "{{ tag }}"
```

```yaml
tasks:

  - name: Install Python dependencies for Docker modules
    pip:
      name:
        - docker
        - requests
      state: present

  - name: Pull Docker image
    docker_image:
      name: "{{ image_name }}:{{ image_tag }}"
      source: pull

  - name: Remove old container if exists
    docker_container:
      name: calculator
      state: absent
      force_kill: true

  - name: Run container
    docker_container:
      name: calculator
      image: "{{ image_name }}:{{ image_tag }}"
      state: started
      restart_policy: always
      tty: true
      interactive: true
```

Inventory File(inventory.ini)

```ini
inventory.ini
[local]
localhost ansible_connection=local ansible_python_interpreter=/Users/veeralagarwal/Documents/MTECH_2/SPE/Calci/Calculator/ansible-env/bin/python ansible_become_pa
```

Screenshot : Ansible Deployment Output

```
Veerals-MacBook-Air:Calculator veeralagarwal$ ansible-playbook deploy.yml -i inventory.ini

PLAY [Deploy Calculator] ***********************************************************************************************************************

TASK [Gathering Facts] *************************************************************************************************************************
ok: [localhost]

TASK [Install Python dependencies for Docker modules] ******************************************************************************************
ok: [localhost]

TASK [Pull Docker image] ***********************************************************************************************************************
ok: [localhost]

TASK [Remove old container if exists] ******************************************************************************************************
ok: [localhost]

TASK [Run container] ***************************************************************************************************************************
changed: [localhost]

PLAY RECAP *************************************************************************************************************************************
localhost                  : ok=5    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

9.Applications Screenshots

Screenshot: CLI Interface & Sample Calculations

```
Veerals-MacBook-Air:Calculator veeralagarwal$ docker run -it chahak02/calculator

Enter 1 :Square Root (√n)
Enter 2 : Factorial (n!)
Enter 3 : Natural log (ln(n))
Enter 4 : Power Function (n^a)
Enter 0 : Exit
Enter Your Choice
1
.....Calculating Square Root......
Enter a number
121
11.0

Enter 1 :Square Root (√n)
Enter 2 : Factorial (n!)
Enter 3 : Natural log (ln(n))
Enter 4 : Power Function (n^a)
Enter 0 : Exit
Enter Your Choice
2
.....Calculating Factorial......
Enter a number
0
1

Enter 1 :Square Root (√n)
Enter 2 : Factorial (n!)
Enter 3 : Natural log (ln(n))
Enter 4 : Power Function (n^a)
Enter 0 : Exit
Enter Your Choice
0
```

---

10. Conclusion

This project demonstrates how to create and implement a '**Scientific Calculator'** using DevOps practices.The integration of GitHub, Maven ,Jenkins, Docker and Ansible ensures efficient and automated software delivery. The project guarantees that the application can be built, tested, packaged, and deployed consistently by incorporating automation into various phases of the

software development lifecycle.The development process is more efficient due to automated pipeline, which also reduces the possibility of human error and manual interactability.

Links:

- GitHub Repository: github.com/Chahak02/SPE_calculator
- Docker Repository: https://hub.docker.com/r/chahak02/calculator

---

Submitted By:

Name: Chahak Agarwal

Roll Number: MT2025031